Applied Software Development Lab
Lab Handout 3: Blinking LED – Bare-Metal C

Learning objectives

- Introduction to programming in C/C++ instead of assembly language
- Interpreting and analyzing the assembly code generated by the C compiler
- Using the oscilloscope to analyze digital output signals

Step 1: Working with the terminal and experimental setup

Goal: Building on the last lab session: we recap how we edit programs, compile them, and transfer them to the microcontroller. We also check the blinking frequency of the LED with the oscilloscope.

As usual, we work with two terminals side by side – the left one for editing, the right one for running commands.

Left terminal (editing)

A) **Clone the Git repository**
Download the prepared code base with the following command:
`git clone https://gitlab.uni-ulm.de/mlehn/avr-bare-metal-2.git`
Among other things, the repository contains the file `led_pin2_delay4.s`, an assembly program that makes an LED connected to pin 2 blink at 1 Hz.

B) **Change into the subdirectory and open the file in nano**
Change into the directory with the assembly code:
`cd avr-bare-metal-2/asm` and open `led_pin2_delay4.s` in `nano`.

Right terminal (compiling & flashing)

A) **Change into the same directory and flash**
Here, too, first change into the subdirectory:
`cd avr-bare-metal-2/asm`
Connect your microcontroller and make it visible to WSL with `usb_check`. Then assemble and upload the code with:
`make upload HEX=led_pin2_delay4.hex`

Task

1. Connect an LED with a suitable series resistor to pin 2 and watch it blink.
2. Measure with the oscilloscope at which frequency the LED actually blinks.
3. Explain and demonstrate at least the following points:
   (a) How does the assembly program make pin 2 oscillate at 1 Hz?
   (b) Which tool translates the assembly code into machine code?
   (c) What does the machine code that is flashed look like?
   (d) Which tool is used to flash the microcontroller?

Step 2: Modifications to the assembly program

Investigate how different assembly instructions affect the behavior of the program – in particular the frequency at pin 2.
You will modify the program step by step and check each time with the oscilloscope whether the LED still blinks at 1 Hz.

Why this matters:

- **Officially:** You will get to know more instructions of the AVR architecture.
- **Unofficially:** You will create variants of assembly code such as a C compiler could also produce. This way, you will later see with your own eyes:
  - **C code can be translated into different assembly code.**
  - **Assembly code is translated into machine code** – and unambiguously so.
  - Conclusion: C is more convenient, but less precise than assembly language.

```
 1          ldi     r19,    4           ; for pin 2 (4 = 2^2)
 2          out     0x0a,   r19
 3          ldi     r18,    0
 4 loop:    eor     r18,    r19         ; 1 cycle
 5          out     0x0b,   r18         ; 1 cycle
 6          ldi     r24,    0x54        ; 1 cycle
 7          ldi     r25,    0x58        ; 1 cycle
 8          ldi     r26,    0x14        ; 1 cycle
 9          ldi     r27,    0x00        ; 1 cycle
10 delay:   subi    r24,    1           ; 1 cycle
11          sbci    r25,    0           ; 1 cycle
12          sbci    r26,    0           ; 1 cycle
13          sbci    r27,    0           ; 1 cycle
14          brne    delay               ; 2 cycles if taken, 1 otherwise
15          rjmp    loop                ; 2 cycles
```

Tasks (measure with the oscilloscope after each step)

1. Replace the following two lines:
   ```
   10 delay:  subi    r24,    1           ; 1 cycle
   11         sbci    r25,    0           ; 1 cycle
   ```
   with this single line:
   ```
   10         sbiw    r24,    1           ; 2 cycles
   ```
   The instruction `sbiw` *(Subtract Immediate from Word)* treats the register pair r25:r24 as a 16-bit value and subtracts 1 here. Although it is not faster (also 2 cycles), the code becomes 2 bytes smaller – because sbiw occupies only one instruction (2 bytes) instead of two (2 × 2 bytes).

2. Replace the following two lines:
   ```
   12         sbci    r26,    0           ; 1 cycle
   13         sbci    r27,    0           ; 1 cycle
   ```
   with:
   ```
   12         sbc     r26,    r1          ; 1 cycle
   13         sbc     r27,    r1          ; 1 cycle
   ```
   The instruction `sbc` *(Subtract with Carry)* works like `sbci` *(Subtract Immediate with Carry)*, except that the second operand here is a register instead of an immediate value. Register `r1` is automatically initialized with 0 on every reset. As long as you do not change it yourself, it therefore always contains the value 0.
   Conclusion: Functionally, and in terms of length and clock cycles, the code is identical, but you learn an alternative way of subtracting the carry bit.
3. Replace the following three lines:
   ```
   1          ldi     r19,    4           ; for pin 2 (4 = 2^2)
   2          out     0x0a,   r19
   3          ldi     r18,    0
   ```
   with:
   ```
   1          ldi     r24,    4           ; for pin 2 (4 = 2^2)
   2          out     0x0a,   r24
   3          ldi     r18,    0
   4          ldi     r19,    4
   ```
   The program works as before – explain why this change is nevertheless not really a good idea.

© Michael Lehn | CC BY-SA 4.0

Step 3: Bare-metal C

In the subdirectory `c` of the repository, you will find C programs for the microcontroller.
To get there from the current directory `asm`, you have to go “one directory up and then one down”:

```
cd ..
cd c
```

Or directly in one go with: `cd ../c`

In the left terminal, open the file `led_pin2_simple.c`:

```
 1 static void setPinModes(unsigned char mode)
 2 {
 3     *(volatile unsigned char *)(0x2A) = mode;
 4 }
 5
 6 static void digitalWriteAll(unsigned char val)
 7 {
 8     *(volatile unsigned char *)(0x2B) = val;
 9 }
10
11 static void delay_cycles(unsigned long count)
12 {
13     do {
14         // the compiler should use a register for count
15         asm volatile("" : "+r"(count));
16         count = count - 1;
17     } while (count != 0);
18 }
19
20 int main()
21 {
22     setPinModes(4);
23     digitalWriteAll(4);
24     while (1) {
25     }
26 }
```

A few words about what you see there:

- In the upper part (lines 1 to 18), three functions are defined: `setPinModes`, `digitalWriteAll`, and `delay_cycles`.
  You do not need to understand the implementation of these functions in detail – at first glance, it may even look daunting, even to hard-boiled C programmers. But of course I am happy to explain the details to you if you are curious.
- In the lower part (lines 20 to 26), you see the function `main`. When the program starts, the statements inside the curly braces are executed one after the other from top to bottom.
  Here, for example, `setPinModes(4)` and `digitalWriteAll(4)` are called – functions with which the pins 0 to 7 can be manipulated.
  What specifically happens:

  - `setPinModes(4)` sets the data direction register so that only pin 2 is configured as an output. (4 corresponds to `00000100` in binary, i.e., only bit 2 is set.)
  - `digitalWriteAll(4)` then sets pin 2 HIGH, because here, too, only bit 2 is set in the argument. The other pins remain LOW.
  - After the output pin has been set with `digitalWriteAll(4)`, the microcontroller should simply do nothing else – it should keep the state permanently.
    The loop

    ```
    24     while (1) {
    25     }
    ```

    makes sure that the program does not end but stays in an endless loop. This is common for microcontrollers: a program usually runs as long as the device is powered – and `main()` should therefore never return, i.e., never terminate.

© Michael Lehn | CC BY-SA 4.0

Task

1. **Compiling and flashing**
   This directory (c) also contains a Makefile that can be used as usual. In the right terminal, run the following command to compile the program and flash it to the microcontroller:
   `make upload HEX=led_pin2_simple.hex`
   a) Connect an LED to pin 2 via a suitable series resistor and make sure that it lights up.

   b) Pay attention to the output of `make`. Explain to us what has changed in the toolchain.
2. **Analyzing automatically generated assembly code**
   Use `make led_pin2_simple.s` to translate the C program into assembly code. Display its content with `cat led_pin2_simple.s`. The generated file contains a lot of additional information (e.g., file name, compiler version, comments) that is not relevant for us. Only a few lines are relevant, for example these:
   ```
           ldi     r24,    lo(4)
           out     0xa,    r24
           out     0xb,    r24
   .L2:
           rjmp    .L2
   ```
   (The lo(4) here simply stands for the constant 4. You could also write ldi r24, 4 directly.)
3. **Writing and comparing your own version**
   In the left terminal, create the file check.s with the following content:
   ```
   1          ldi     r24,    4
   2          out     0xa,    r24
   3          out     0xb,    r24
   4 halt:    rjmp    halt
   ```

   Generate machine code from it with `make check.hex`
   a) Compare the generated machine code with that of the C program:
   `cat led_pin2_simple.hex`
   `cat check.hex`
   Or use the `diff` command: `diff check.hex led_pin2_simple.hex`
   If no differences are found, the output stays empty – otherwise `diff` shows the lines that differ.
   b) In `check.s`, change the first line:
   ```
   1          ldi     r24,    4
   ```
   to
   ```
   1          ldi     r24,    5
   ```
   Assemble it again with `make check.hex` and compare again with the `diff` command. Now a difference should be displayed – check where the generated .hex files differ.

© Michael Lehn | CC BY-SA 4.0

Step 4: Blinking more elegantly with exclusive OR

Modify the function `main()` as follows:

```
20 int main()
21 {
22     setPinModes(4);
23     unsigned char pinValues = 4;
24     while (1) {
25         pinValues = pinValues ^ 4;
26         digitalWriteAll(pinValues);
27     }
28 }
```

Tasks

1. **Testing the modified program**
   Before we analyze in more detail why the program works, we first test whether it works:

   - Compile the program with the Makefile as usual and flash it to the microcontroller with: `make upload HEX=led_pin2_simple.hex`
   - Observe what the LED at pin 2 does.
   - Measure with the oscilloscope how the signal at pin 2 behaves.

2. **Writing and comparing your own version**
   Compile the C program with `make led_pin2_simple.s`. This generates the assembly code produced by the compiler. Display its content with: `cat led_pin2_simple.s`
   The part of the generated code that is relevant for us is equivalent to:

   ```
   1         ldi     r24,    4
   2         out     0xa,    r24
   3         ldi     r25,    4
   4 loop:   eor     r24,    r25
   5         out     0xb,    r24
   6         rjmp    loop
   ```

   In the left terminal, create the file `check.s` with exactly this content. Build both programs with: `make led_pin2_simple.hex check.hex` and compare the generated machine code with `diff`.

Brief explanation of the C code

In line 23,

```
23     unsigned char pinValues = 4;
```

a **local** variable of type `unsigned char` is defined and initialized with 4:

- The type `unsigned char` specifies that the variable occupies exactly one byte.
- Since pinValues was defined inside the function `main` (without special additional keywords), it is a local variable with automatic storage duration: it exists only while the function is executing and is no longer valid after the function has ended.
  The compiler may place such **local** variables in registers, which is particularly efficient. If, on the other hand, a variable is defined outside all functions, it is **global** and permanently requires memory in RAM.

In line 25,

```
25         pinValues = pinValues ^ 4;
```

the current value of pinValues is replaced by its exclusive OR with 4.

As you know, this has the effect that bit 2 is toggled (flipped):

- if bit 2 was 1 before → it becomes 0,
- if it was 0 → it becomes 1.

This creates the blinking effect.

© Michael Lehn | CC BY-SA 4.0

Step 5: Calmer blinking at 1 Hz

Great – once again we have a program that switches **pin 2 alternately HIGH for about 250 ns and then LOW**.
This time, however, the program is **written in C**, translated by the **C compiler into assembly code**, and then by the **assembler into machine code**. So, in a roundabout way, we have achieved the same as before – but with more modern means.

Even if the C code (even main() alone) looks cryptic to a layperson at first, you can sense that it is considerably **easier to learn the rules of C** than to program directly in assembly language or even machine code – especially for larger projects.

In the last example, the LED should blink at 1 Hz again. To do this, we insert a delay at the right place in the loop that waits for half a second each time – this results in a complete cycle of one second (1 Hz). The required number of clock cycles can be calculated:

$$n = \frac{\frac{500000000}{62.5} - 7}{6} = 1333332 \text{ cycles}$$

In the C program, modify the function main() as follows:

```
20 int main()
21 {
22     // setup
23     setPinModes(4);
24
25     unsigned char pinValues = 0;
26     while (1) {
27         // loop
28         pinValues = pinValues ^ 4;
29         digitalWriteAll(pinValues);
30         delay_cycles((500000000 / 62.5 - 7) / 6);
31     }
32 }
```

Task

1. Make the LED blink at 1 Hz.
2. Run `diff led_pin2_simple.hex ../asm/led_pin2_delay4.hex` and convince yourself that the machine code is identical to what we wrote in assembly language in Step 2.

© Michael Lehn | CC BY-SA 4.0